

DevOps CA-II Report:

DevOps Pipeline for Virtual Zoo

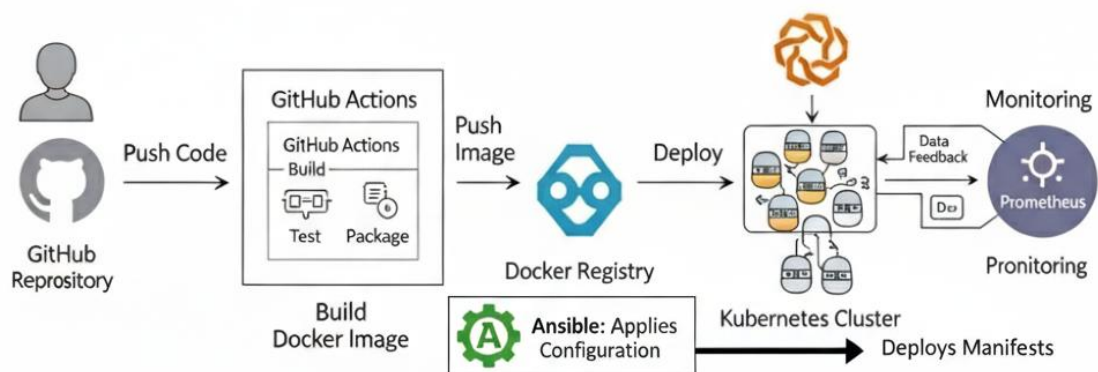
Group Members:

Name	PRN	Batch
Archit Patil	22070122025	Th1
Patoliya Shyam Gopal	22070122141	Th2
Ritam Mishra	22070122165	Th2
Vedant Ambadkar	22070122247	Th2

1. Architecture

The final architecture consists of a containerized web application deployed on a Kubernetes cluster, with a full CI/CD pipeline for automation and a monitoring setup.

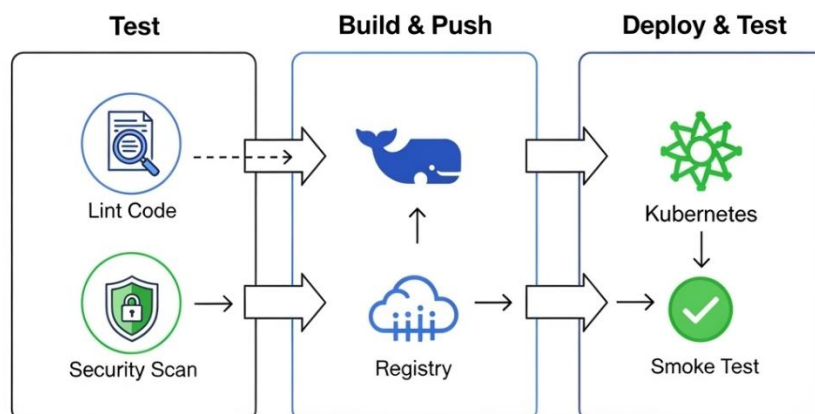
- **Application:** A simple static website (HTML, CSS, JS).
- **Containerization:** The application is packaged into a Docker image using Nginx as the web server. This image is stored in Docker Hub.
- **Orchestration:** **Kubernetes** is used to manage the application's lifecycle.
 - A Deployment ensures a desired number of application pods are always running.
 - A Service (LoadBalancer) exposes the application to be accessed.
- **Monitoring:** **Prometheus** is deployed within the cluster to scrape and store metrics from the application, providing visibility into its performance and health.



2. Pipeline Flow (CI/CD)

The entire workflow is automated using GitHub Actions. The pipeline is triggered on a git push to the main branch and is structured into distinct, dependent jobs.

1. **Trigger:** Developer pushes code changes to the GitHub repository.
2. **Test Job:**
 - **Linting:** The HTML code is checked for syntax and style errors using htmlhint.
 - **Security Scan:** The Dockerfile is used to build the image, which is then scanned for known vulnerabilities (CVEs) using Trivy. The job fails if high-severity issues are found.
3. **Build & Push Job:**
 - If the test job succeeds, this job runs.
 - It builds the final Docker image.
 - It logs into Docker Hub using secrets and pushes the newly tagged image to the registry.
4. **Deploy & Test Job:**
 - If the build-and-push job succeeds, this job begins.
 - **Environment Provisioning:** A temporary, clean Kubernetes cluster is created inside the runner using kind (Kubernetes in Docker).
 - **Configuration Management:** **Ansible** is used to apply the Kubernetes manifests (deployment.yaml, service.yaml) to the cluster.
 - **Smoke Test:** A simple curl command is run against the deployed application's service to verify that it is running and responding with a success code.



3. Challenges Encountered

- **Local vs. CI Environment Mismatch:** The initial kubeconfig from Docker Desktop used the local-only address `kubernetes.docker.internal`, causing DNS failures in the cloud-based GitHub Actions runner. This highlighted the critical difference between local and CI environments.
- **Implicit Dependencies in CI:** The pipeline failed multiple times due to missing dependencies in the runner.
 - The `html-lint` tool required `phantomjs`, an uninstalled dependency.
 - Ansible required the `kubernetes.core` collection, which had to be explicitly installed with `ansible-galaxy`.
- **Ephemeral Nature of CI Jobs:** The smoke test initially failed because it was in a separate job from the deployment. We learned that each job runs in a new, isolated environment, and the kind cluster created in the deploy job was destroyed before the smoke-test job could run.

4. Lessons Learned

- **Environment Parity is Crucial:** Configuration must be portable. Relying on local-specific settings (kubeconfig) is a common pitfall. The solution is to use cloud-based clusters with public endpoints for CI/CD or to create a fully self-contained environment within the runner (`kind`).
- **CI Runners are Minimal and Ephemeral:** Never assume a tool or dependency is pre-installed. Every requirement—from Ansible collections to system binaries—must be explicitly set up in the workflow. State is not preserved between jobs.
- **The Value of Layered Testing:** Our final pipeline incorporates multiple forms of testing.
 - Static Analysis (Linting): Catches code quality issues instantly.
 - Security Scanning (Trivy): Prevents shipping code with known vulnerabilities.
 - Smoke Testing: Provides confidence that the application is not fundamentally broken after deployment.
- **Choose Modern, Well-Maintained Tools:** The initial linter (`html-lint`) depended on an unmaintained tool (`phantomjs`). Switching to a modern alternative (`htmlhint`) resolved the issue and resulted in a cleaner, more reliable pipeline.